UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM

APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

MODUL 8 – Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana)

Nama Mahasiswa : Nailah Azka Iftinah
NIM : 23051430026
Kelas : A
Tanggal Praktikum : 17 April 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 8 (Pertemuan 8)
Topik Utama	Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana)
Sub-Topik	Konsep Asynchronous, Fetch API, Promise, Async/Await, dan integrasi REST API eksternal.
Durasi Praktikum	340 Menit
Tanggal Praktikum	17 April 2026
Nama Mahasiswa	Nailah Azka Iftinah
NIM	23051430026
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami perbedaan kode sinkronus dan asinkronus.
2	Mampu menggunakan `fetch()` untuk mengambil data dari server eksternal (API).
3	Memahami format data JSON (JavaScript Object Notation).
4	Menerapkan penanganan error saat request API gagal.

C. ALAT DAN BAHA

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern (Chrome/Firefox/Edge).	Versi terbaru	Testing & Debugging
3	Ekstensi VS Code	*Live Server & CodeSnap*	Menjalankan HTML di server lokal

			dan memperbarui tampilan otomatis saat file berubah. Mengubah kode program menjadi gambar untuk dokumentasi
5	Git/Github	Versi Terbaru	Repository dan Manajemen versi
6	JavaScript (Client-Side)	ES6 (Modern JavaScript)	Bahasa pemrograman untuk logika & perhitungan
7	Quokka.js	Extension VS Code	Untuk melihat hasil JavaScript secara realtime

D. DASAR TEORI

Asynchronous JavaScript adalah teknik pemrograman yang memungkinkan browser menjalankan tugas-tugas berat, seperti mengambil data dari server, di latar belakang tanpa menghentikan atau membekukan antarmuka pengguna (UI). Dalam model tradisional atau synchronous, kode dijalankan baris demi baris; jika satu baris membutuhkan waktu lama untuk merespons, seluruh aplikasi akan "macet" (freeze). Pendekatan asinkronus sangat krusial dalam dunia industri karena sistem dashboard atau monitoring produksi seringkali harus mengambil data dari lokasi fisik yang jauh (misal: server pusat atau sensor IoT). Dengan asinkronus, pengguna tetap bisa berinteraksi dengan aplikasi sementara data sedang diunduh di balik layar.

Fetch API adalah antarmuka modern yang disediakan oleh browser untuk melakukan permintaan HTTP (HTTP Request) guna mengambil atau mengirim data ke server. Fungsi fetch() bekerja berdasarkan konsep Promise, yaitu sebuah objek yang mewakili keberhasilan atau kegagalan dari sebuah operasi asinkron. Saat memanggil data dari API eksternal, fetch() akan memberikan respons yang perlu diperiksa status keberhasilannya (misal: status 200 untuk sukses). Penggunaan Fetch API mempermudah integrasi aplikasi web dengan berbagai layanan data tanpa perlu memuat ulang seluruh halaman (Single Page Application).

JSON (JavaScript Object Notation) adalah format pertukaran data berbasis teks yang ringan dan mudah dibaca oleh manusia maupun mesin. Di era Industri 4.0, JSON telah menjadi standar de-facto untuk komunikasi antara aplikasi web dengan database atau sistem ERP. Dalam alur kerja API, data yang diterima dari server biasanya berbentuk string JSON yang perlu dikonversi menjadi objek JavaScript menggunakan metode .json() agar propertinya dapat diakses dan ditampilkan ke dalam elemen HTML seperti tabel atau kartu laporan.

Async/Await adalah sintaksis modern dalam JavaScript yang membuat penulisan kode asinkron menjadi lebih bersih dan terlihat seperti kode sinkron sehingga lebih mudah dibaca dan dipelihara. Kata kunci async digunakan untuk mendeklarasikan fungsi asinkron, sementara await digunakan untuk menunda eksekusi hingga Promise selesai (berhasil atau gagal). Untuk menangani kesalahan dalam blok ini, digunakan struktur try...catch. Pendekatan ini sangat direkomendasikan dalam pengembangan aplikasi skala besar untuk memastikan bahwa error, seperti kegagalan koneksi internet atau data tidak ditemukan, dapat ditangani dengan elegan tanpa merusak fungsionalitas aplikasi lainnya.

REST API (Representational State Transfer) adalah arsitektur komunikasi yang menggunakan protokol HTTP untuk melakukan operasi data. Operasi ini biasanya mencakup metode GET untuk mengambil data (seperti melihat daftar inventaris), POST untuk mengirim data baru (seperti melaporkan insiden produksi), PUT untuk memperbarui data, dan DELETE untuk menghapus data. Dengan memanfaatkan REST API seperti JSONPlaceholder dalam praktikum ini, mahasiswa dapat mensimulasikan interaksi nyata antara aplikasi front-end dengan back-end server yang menyimpan database perusahaan.

D.1 Konsep Utama

Pada praktikum pertemuan kedelapan ini, pembelajaran memasuki tahap integrasi data luar, yaitu membuat aplikasi web yang mampu berkomunikasi dengan server melalui jaringan internet. Jika pada pertemuan sebelumnya data hanya dikelola secara lokal di dalam browser, pada pertemuan ini diperkenalkan mekanisme Asynchronous JavaScript menggunakan Fetch API. Hal ini memungkinkan aplikasi untuk mengambil data dari sumber eksternal secara real-time tanpa harus memuat ulang seluruh halaman, yang merupakan standar pengembangan aplikasi dashboard industri modern.

Konsep pertama yang dipelajari adalah perbedaan antara Synchronous dan Asynchronous. Dalam model synchronous, kode dijalankan berurutan; jika sebuah permintaan data ke server memakan waktu lama, seluruh tampilan web akan membeku (freeze). Sebaliknya, dengan model asynchronous, browser dapat menjalankan tugas di latar belakang, seperti mengunduh data stok Gudang, sambil tetap mengizinkan pengguna

berinteraksi dengan menu lainnya. Hal ini sangat krusial bagi kenyamanan pengguna (User Experience) agar aplikasi tetap responsif meskipun koneksi internet sedang lambat.

Konsep kedua adalah Fetch API dan Promise. fetch() adalah fungsi modern JavaScript yang digunakan untuk mengirim permintaan HTTP ke server. Fungsi ini menghasilkan sebuah Promise, yaitu objek yang menjanjikan hasil di masa depan: apakah permintaan tersebut berhasil atau gagal. Saat melakukan fetch(), kita harus menangani respon dari server, memeriksa status keberhasilannya, dan kemudian memproses data tersebut. Kemampuan ini menjadi fondasi bagi mahasiswa untuk menghubungkan aplikasi mereka dengan database terpusat atau sensor-sensor IoT di area pabrik.

Konsep ketiga adalah JSON (JavaScript Object Notation) sebagai standar pertukaran data. Saat aplikasi melakukan fetch ke server, data yang dikirimkan biasanya dalam format JSON, yaitu teks terstruktur yang ringan. Karena data tersebut datang dalam bentuk string teks, aplikasi harus melakukan parsing menggunakan metode .json() agar string tersebut berubah kembali menjadi objek JavaScript yang bisa diolah. Pemahaman format JSON sangat penting karena hampir semua API modern, mulai dari data karyawan hingga log produksi, menggunakan format ini untuk berkomunikasi antar sistem yang berbeda.

Konsep keempat adalah Async/Await dan Error Handling. Async/Await adalah cara penulisan kode asinkron agar terlihat lebih sederhana dan mudah dibaca seperti kode biasa. Dalam proses ini, kita menggunakan blok try...catch untuk menangani kemungkinan error, seperti ketika kabel internet terputus atau server tujuan sedang down. Penanganan error yang baik memastikan aplikasi tidak langsung rusak (crash) saat terjadi gangguan jaringan, melainkan dapat memberikan pesan peringatan yang informatif kepada operator atau pengguna.

Konsep kelima adalah Metode HTTP (GET dan POST). Dalam integrasi API, terdapat berbagai cara berinteraksi dengan data. Metode GET digunakan untuk mengambil atau membaca data dari server (seperti menampilkan daftar laporan insiden), sedangkan metode POST digunakan untuk mengirimkan data baru ke server (seperti menambah data karyawan baru). Dengan memahami kedua metode ini, mahasiswa dapat mensimulasikan alur kerja penuh sebuah sistem informasi, di mana data tidak hanya bisa dibaca, tetapi juga bisa dikirim dan disimpan ke server pusat melalui protokol web standar.

Terakhir adalah Alur Kerja Integrasi Data. Setiap fitur yang dibuat pada praktikum ini mengikuti pola yang sistematis: aplikasi melakukan permintaan ke URL API, browser menunggu respon di latar belakang, setelah data diterima kemudian difilter atau diproses sesuai kebutuhan, dan terakhir elemen HTML (seperti kartu atau tabel) diperbarui secara

dinamis menggunakan DOM Manipulation. Pola ini menyerupai sistem monitoring di industri di mana data dari berbagai departemen ditarik secara otomatis untuk disajikan dalam satu dashboard manajemen yang terintegrasi.

D.2 Keterkaitan dengan Teknik Industri

Jika dikaitkan dengan Teknik Industri, materi pada pertemuan kedelapan ini sangat krusial karena memperkenalkan konsep Integrasi Sistem dan Konektivitas Data yang menjadi tulang punggung Industri 4.0. Di lingkungan manufaktur modern, data tidak lagi berdiri sendiri di satu komputer, melainkan tersebar di berbagai titik seperti server pusat, database ERP (Enterprise Resource Planning), hingga sensor-sensor pada mesin di lantai produksi. Materi Asynchronous JavaScript dan Fetch API yang dipelajari pada praktikum ini merupakan fondasi teknis tentang bagaimana sebuah dashboard manajerial menarik data real-time dari berbagai sumber eksternal tersebut untuk disajikan sebagai informasi pendukung keputusan.

Konsep Asynchronous (Asinkronus) dapat dianalogikan sebagai sistem aliran material yang fleksibel di sebuah pabrik. Dalam sistem Synchronous, jika satu stasiun kerja mengalami kendala, maka seluruh lini produksi akan berhenti total (bottleneck). Namun, dengan pendekatan Asynchronous, aplikasi web bertindak seperti sistem yang memiliki buffer; sistem tetap bisa melayani permintaan pengguna (seperti input data atau navigasi menu) sambil menunggu kiriman data dari server di latar belakang. Dalam Teknik Industri, hal ini mencerminkan prinsip efisiensi dan responsivitas sistem, di mana sebuah gangguan pada proses pengambilan data tidak boleh menyebabkan seluruh sistem pemantauan menjadi lumpuh atau tidak bisa digunakan oleh operator.

Penggunaan JSON dan REST API mencerminkan standar komunikasi data universal dalam integrasi sistem industri. Sebuah pabrik seringkali memiliki mesin dari berbagai vendor yang berbeda, dan JSON bertindak sebagai bahasa universal yang memungkinkan mesin-mesin tersebut berkomunikasi dengan sistem pemantauan terpusat. Ketika mahasiswa mempraktikkan pengambilan data karyawan atau laporan insiden dari API eksternal, mereka sebenarnya sedang mensimulasikan cara kerja sistem MES (Manufacturing Execution System) yang menarik data dari database server untuk memonitor efektivitas peralatan secara keseluruhan atau Overall Equipment Effectiveness (OEE).

Operasi GET dan POST melalui Fetch API mencerminkan siklus pertukaran informasi dalam manajemen rantai pasok digital. Metode GET mencerminkan aktivitas Monitoring atau penarikan data stok bahan baku dari gudang pusat ke dashboard produksi, sedangkan metode POST mencerminkan aktivitas Reporting atau pengiriman laporan output produksi

harian dari lantai pabrik kembali ke database server perusahaan. Pemahaman terhadap metode ini memberikan kemampuan bagi mahasiswa Teknik Industri untuk merancang aplikasi yang tidak hanya mampu membaca data, tetapi juga mampu memberikan umpan balik (feedback) ke sistem informasi perusahaan secara otomatis dan akurat.

Terakhir, penguasaan materi ini membekali mahasiswa dengan kompetensi untuk membangun sistem yang terkoneksi dan terintegrasi. Di era Industri 4.0, seorang Industrial Engineer diharapkan mampu menjembatani kesenjangan antara proses fisik di pabrik dengan sistem digital di kantor pusat. Dengan memahami bagaimana cara mengambil data melalui API dan menanganinya secara asinkron, mahasiswa dapat menciptakan sistem dashboard yang dinamis, cepat, dan selalu terbarui sesuai dengan kondisi nyata di lapangan. Hal ini sangat penting untuk mendukung prinsip Real-Time Capability dalam manajemen industri modern, di mana kecepatan akses terhadap data yang valid menjadi kunci dalam memenangkan persaingan di pasar.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

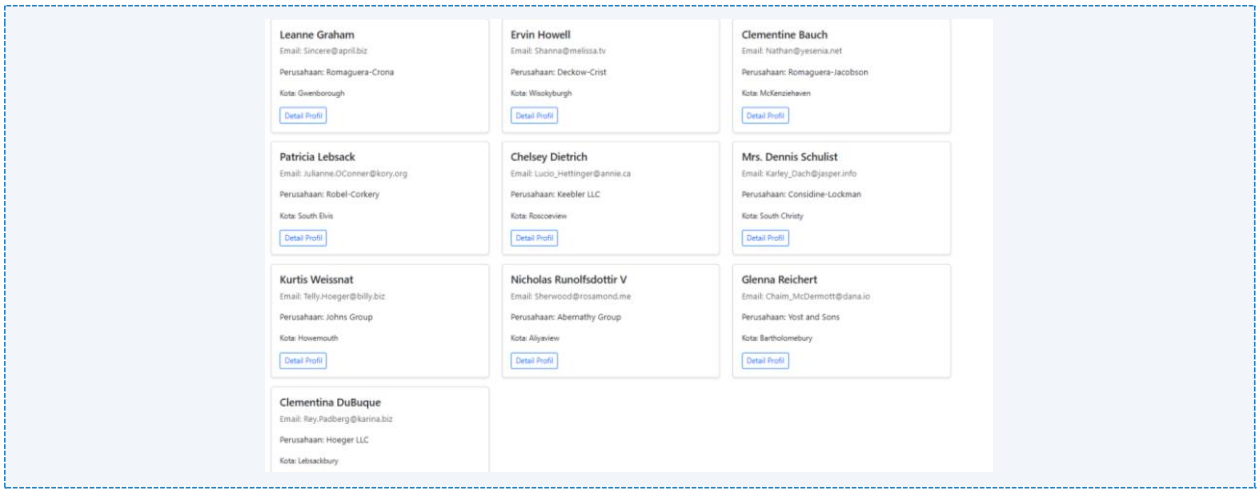
Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1 – Mengenal JSONPlaceholder

Kita akan menggunakan layanan gratis `https://jsonplaceholder.typicode.com/users` sebagai simulasi database "Karyawan" atau "Agen Supplier".

```
/* ======= KODE / OUTPUT LANGKAH 1 HTML=======
const API_URL = 'https://jsonplaceholder.typicode.com/users';
```

Screenshot Hasil:



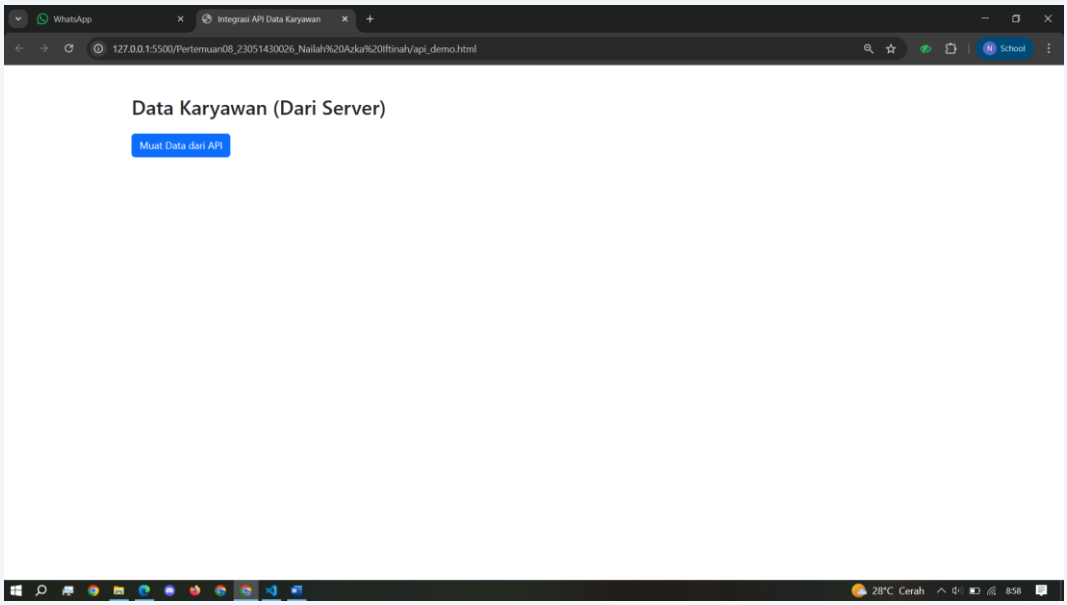
Gambar E.3– Hasil Logika JavaScript (Simpan & Baca)

E.2 Langkah 2 – Membuat HTML untuk Menampilkan Data

Buat file `api_demo.html`.

```
/* ======= KODE / OUTPUT LANGKAH 2 HTML=======
<!DOCTYPE html>
<html lang="id">

<head>
    <meta charset="UTF-8">
    <title>Integrasi API Data Karyawan</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="container mt-5">
    <h2 class="mb-4">Data Karyawan (Dari Server)</h2>

    <div class="mb-3">
        <button id="btnLoad" class="btn btn-primary">Muat Data dari
API</button>
        <!-- Membuat btnTambah untuk menambah karyawan baru(Latihan 1) -->
        <button id="btnTambah" class="btn btn-success">Tambah Karyawan
Baru</button>
        <div id="loading" class="spinner-border text-primary d-none"
role="status"></div>
    </div>
    <div class="row" id="containerKaryawan">
        <!-- Card karyawan akan muncul di sini -->
    </div>
    <script src="api_script.js"></script>
</body>

</html>
```

Screenshot Hasil:



Gambar E.2 – Membuat HTML untuk Menampilkan Data

E.3 Langkah 3 – Fetch Data dengan .then() (Promise)

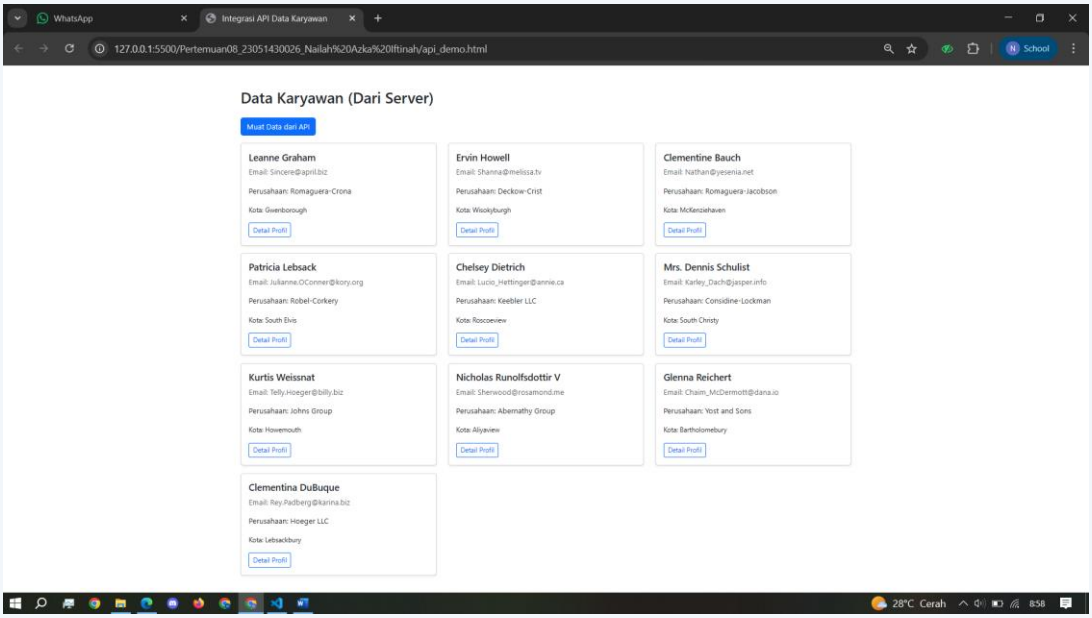
Buat `api_script.js`.

```
/* ======= KODE / OUTPUT LANGKAH 2 ======= */
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

// Endpoint API (Simulasi Database)
const API_URL = 'https://jsonplaceholder.typicode.com/users';

btnLoad.addEventListener('click', function () {
    // Tampilkan loading
    loading.classList.remove('d-none');
    container.innerHTML = ''; // Bersihkan konten lama

    // Fetch Data
    fetch(API_URL)
        .then(function (response) {
            // Cek jika response sukses (kode 200-299)
            if (!response.ok) {
                throw new Error('Gagal mengambil data');
            }

            // Parsing data JSON
            return response.json();
        })
        .then(function (dataKaryawan) {
```

```
            // Data berhasil didapat
            console.log(dataKaryawan); // Cek di console browser
            renderData(dataKaryawan);
        })
        .catch(function (error) {
            // Jika ada error (misal putus internet)
            container.innerHTML = `
                <div class="alert alert-danger">
                    Error: ${error.message}
                </div>
            `;
        })
        .finally(function () {
            // Sembunyikan loading (baik sukses maupun gagal)
            loading.classList.add('d-none');
        });
});

function renderData(data) {
    data.forEach(function (karyawan) {
        // Buat elemen card Bootstrap
        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';

        col.innerHTML = `
            <div class="card h-100 shadow-sm">
                <div class="card-body">
                    <h5 class="card-title">${karyawan.name}</h5>
                    <p class="card-text text-muted">
                        Email: ${karyawan.email}
                    </p>
                    <p class="card-text">
                        Perusahaan: ${karyawan.company.name}
                    </p>
                    <p class="card-text">
                        <small>Kota: ${karyawan.address.city}</small>
                    </p>
                    <a href="" class="btn btn-sm btn-outline-primary">
                        Detail Profil
                    </a>
                </div>
            </div>
        `;

        container.appendChild(col);
    });
```

```
}
```

Screenshot Hasil:



Gambar E.3– Fetch Data dengan .then() (Promise)

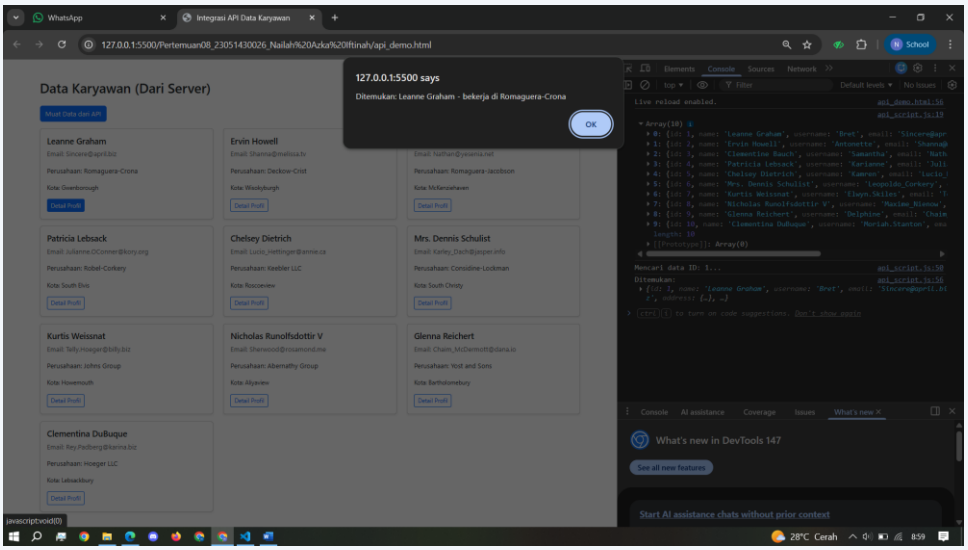
E.4 Langkah 4 – Fetch dengan Async/Await (Modern Style)

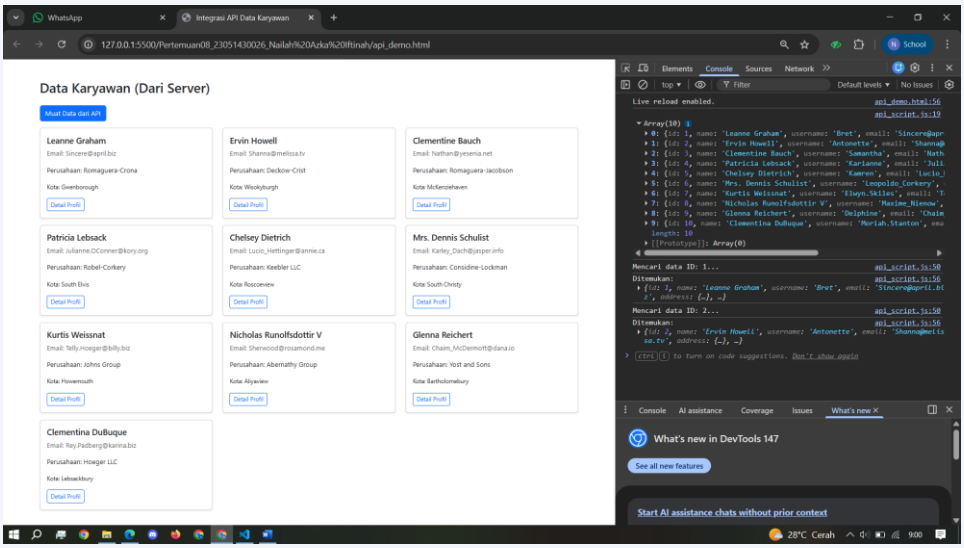
Async/Await membuat kode asinkron terlihat seperti sinkron (lebih mudah dibaca). Mari kita buat fungsi baru untuk mencari karyawan berdasarkan ID. Tambahkan di `api_script.js`:

```
/* ======= KODE / OUTPUT LANGKAH 2 ======= */
// Fungsi Async untuk mencari data spesifik
async function cariKaryawan(id) {
    try {
        console.log(`Mencari data ID: ${id}...`);

        const response = await fetch(
            `https://jsonplaceholder.typicode.com/users/${id}`
        );

        if (!response.ok) {
            throw new Error('Data tidak ditemukan');
        }

        const data = await response.json();
        console.log('Ditemukan:', data);

        alert(
```

```
            `Ditemukan: ${data.name} - bekerja di ${data.company.name}`
        );
    } catch (error) {
        console.error(error);
        alert(error.message);
    }
}

// Contoh pemanggilan: cariKaryawan(2);
// Anda bisa memanggil fungsi ini lewat console browser untuk tes.
```

Screenshot Hasil:





Gambar E.4 – Fetch dengan Async/Await (Modern Style)

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – (Posting Data):

Pertanyaan / Instruksi Latihan:

> Latihan 1 (Posting Data): Buat tombol "Tambah Karyawan Baru". Gunakan method `fetch` dengan opsi `{ method: 'POST', body: ... }` untuk mengirim data dummy ke API. Cek response di console.

Jawaban / Kode Solusi:

```
1. JAWABAN KODE JS

const btnLoad = document.getElementById('btnLoad');
const btnAdd = document.getElementById('btnAdd');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

const API_URL = 'https://jsonplaceholder.typicode.com/users';

// =======================
// GET DATA (LOAD)
// =======================
btnLoad.addEventListener('click', function () {
    loading.classList.remove('d-none');
    container.innerHTML = '';

    fetch(API_URL)
        .then(function (response) {
            if (!response.ok) {
                throw new Error('Gagal mengambil data dari server');
            }
            return response.json();
        })
        .then(function (dataKaryawan) {
            console.log("Data diterima:", dataKaryawan);
            renderData(dataKaryawan);
        })
        .catch(function (error) {
            container.innerHTML = `
                <div class="alert alert-danger">
                    Error: ${error.message}
                </div>
            `;
```

```
        })
        .finally(function () {
            loading.classList.add('d-none');
        });
});


// =======================
// POST DATA (TAMBAH)
// =======================
btnAdd.addEventListener('click', function () {

    const karyawanBaru = {
        name: "Budi Utomo (User Baru)",
        email: "budi.utomo@industri.com",
        company: { name: "PT. Maju Digital" },
        address: { city: "Yogyakarta" }
    };

    console.log("Mengirim data...");
    loading.classList.remove('d-none');

    fetch(API_URL, {
        method: 'POST',
        body: JSON.stringify(karyawanBaru),
        headers: {
            'Content-type': 'application/json; charset=UTF-8',
        },
    })
    .then(response => {
        if (!response.ok) throw new Error('Gagal mengirim data');
        return response.json();
    })
    .then(hasil => {
        console.log("Hasil POST:", hasil);

        alert(
            `Sukses! Karyawan baru berhasil terdaftar secara virtual.\n` +
            `Nama: ${hasil.name}\n` +
            `Email: ${karyawanBaru.email}\n` +
            `Perusahaan: ${karyawanBaru.company.name}\n` +
            `Kota: ${karyawanBaru.address.city}\n` +
            `ID Server: ${hasil.id}`
        );

        // TAMBAH KE TAMPILAN (INI YANG BIKIN MUNCUL)
```

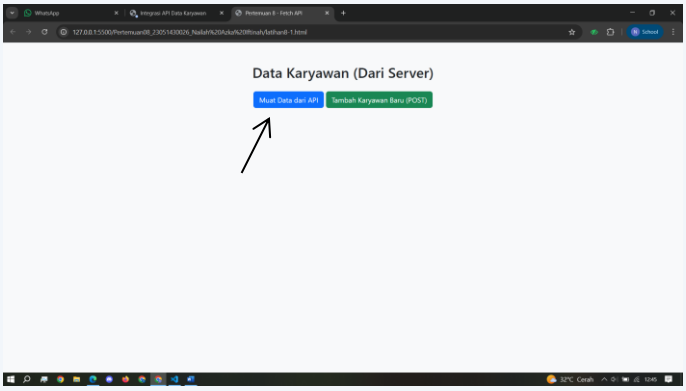
```
        renderData([karyawanBaru]);
    })
    .catch(error => {
        console.error("Error:", error);
        alert("Terjadi kesalahan saat menambah data.");
    })
    .finally(() => {
        loading.classList.add('d-none');
    });
});


// =======================
// RENDER DATA
// =======================
function renderData(data) {
    data.forEach(function (karyawan) {
        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';

        col.innerHTML = `
            <div class="card shadow-sm">
                <div class="card-body">
                    <h5 class="card-title text-primary">${karyawan.name}</h5>
                    <p class="card-text">
                        <strong>Email:</strong> ${karyawan.email}<br>
                        <strong>Perusahaan:</strong>
${karyawan.company.name}<br>
                        <strong>Kota:</strong> ${karyawan.address.city}
                    </p>
                    <button class="btn btn-outline-primary btn-sm">
                        Detail Profil
                    </button>
                </div>
            </div>
        `;

        container.appendChild(col);
    });
}
```

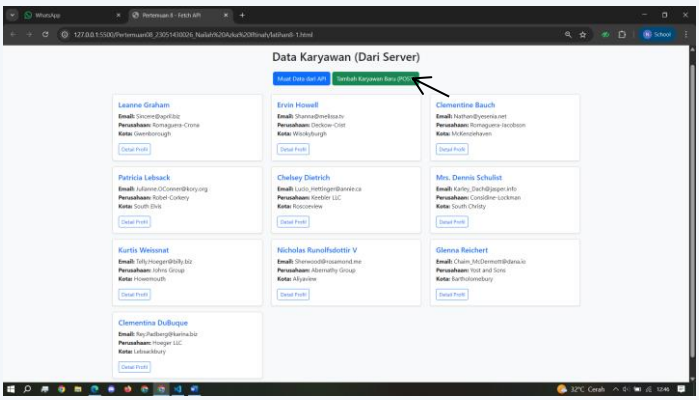
2. JAWABAN KODE HTML

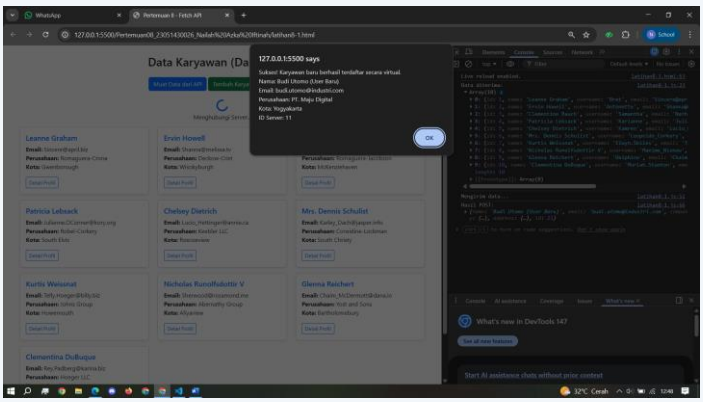
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
```

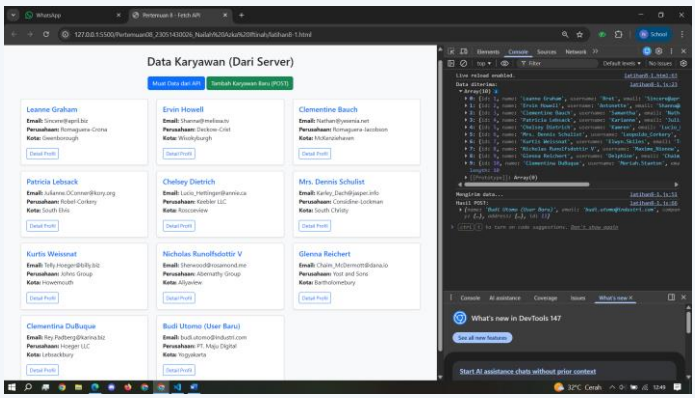
```html
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Pertemuan 8 - Fetch API</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="bg-light">

    <div class="container my-5">
        <h2 class="text-center mb-4">Data Karyawan (Dari Server)</h2>

        <div class="text-center mb-4">
            <button id="btnLoad" class="btn btn-primary">
                Muat Data dari API
            </button>

            <button id="btnAdd" class="btn btn-success">
                Tambah Karyawan Baru (POST)
            </button>
        </div>

        <div id="loading" class="d-none text-center my-3">
            <div class="spinner-border text-primary" role="status"></div>
            <p>Menghubungi Server...</p>
        </div>

        <div id="containerKaryawan" class="row"></div>
    </div>

    <script src="latihan8-1.js"></script>

</body>
</html>
```

Screenshot Hasil:









Gambar F.1 – Hasil Latihan 1

F.2 Latihan 2 – (Sortir):

Pertanyaan / Instruksi Latihan:

> Latihan 2 (Sortir): Tambahkan tombol "Sortir berdasarkan Jumlah (Terbesar)" yang mengurutkan tabel dari jumlah produksi terbanyak ke terkecil.

Jawaban / Kode Solusi:

```
1. JAWABAN KODE JS
```

```js
const btnLoad = document.getElementById('btnLoad');
const btnAdd = document.getElementById('btnAdd');
const btnFilter = document.getElementById('btnFilter');
const btnReset = document.getElementById('btnReset');
const inputFilter = document.getElementById('inputFilter');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

const API_URL = 'https://jsonplaceholder.typicode.com/users';

let semuaData = [];


// =======================
// GET DATA (LOAD)
// =======================
btnLoad.addEventListener('click', function () {
    loading.classList.remove('d-none');
    container.innerHTML = '';

    fetch(API_URL)
        .then(function (response) {
            if (!response.ok) {
                throw new Error('Gagal mengambil data dari server');
            }
            return response.json();
        })
        .then(function (dataKaryawan) {
            console.log("Data diterima:", dataKaryawan);
            semuaData = dataKaryawan;
            renderData(semuaData);
        })
        .catch(function (error) {
            container.innerHTML = `
                <div class="alert alert-danger">
                    Error: ${error.message}
                </div>
```

```
            `;
        })
        .finally(function () {
            loading.classList.add('d-none');
        });
});


// =======================
// POST DATA (TAMBAH)
// =======================
btnAdd.addEventListener('click', function () {

    const karyawanBaru = {
        name: "Budi Utomo (User Baru)",
        email: "budi.utomo@industri.com",
        company: { name: "PT. Maju Digital" },
        address: { city: "Yogyakarta" }
    };

    loading.classList.remove('d-none');

    fetch(API_URL, {
        method: 'POST',
        body: JSON.stringify(karyawanBaru),
        headers: {
            'Content-type': 'application/json; charset=UTF-8',
        },
    })
    .then(function (response) {
        if (!response.ok) throw new Error('Gagal mengirim data');
        return response.json();
    })
    .then(function (hasil) {
        console.log("Hasil POST:", hasil);

        alert(
            `Sukses! Karyawan baru berhasil terdaftar secara virtual.\n` +
            `Nama: ${hasil.name}\n` +
            `Email: ${karyawanBaru.email}\n` +
            `Perusahaan: ${karyawanBaru.company.name}\n` +
            `Kota: ${karyawanBaru.address.city}\n` +
            `ID Server: ${hasil.id}`
        );

        semuaData.push(karyawanBaru);
```

```
        renderData([karyawanBaru]);
    })
    .catch(function (error) {
        console.error("Error:", error);
        alert("Terjadi kesalahan saat menambah data.");
    })
    .finally(function () {
        loading.classList.add('d-none');
    });
});


// =======================
// FILTER KOTA (SEARCH)
// =======================
btnFilter.addEventListener('click', function () {
    const keyword = inputFilter.value.toLowerCase().trim();

    if (keyword === '') {
        alert('Masukkan kata kunci filter terlebih dahulu.');
        return;
    }

    if (semuaData.length === 0) {
        alert('Data belum dimuat. Klik "Muat Data dari API" terlebih
dahulu.');
        return;
    }

    // filter() sebelum forEach sesuai instruksi Latihan 2
    const dataFilter = semuaData.filter(function (karyawan) {
        return karyawan.address.city.toLowerCase().includes(keyword);
    });

    console.log(`Hasil filter kota mengandung "${keyword}":`, dataFilter);

    container.innerHTML = '';

    if (dataFilter.length === 0) {
        container.innerHTML = `
            <div class="col-12">
                <div class="alert alert-warning">
                    Tidak ada karyawan dengan kota yang mengandung
"<strong>${keyword}</strong>".
                </div>
            </div>
```

```
        `;
        return;
    }

    renderData(dataFilter);
});

// Tekan Enter di input juga bisa trigger filter
inputFilter.addEventListener('keydown', function (e) {
    if (e.key === 'Enter') btnFilter.click();
});


// =======================
// RESET FILTER
// =======================
btnReset.addEventListener('click', function () {
    inputFilter.value = '';
    container.innerHTML = '';

    if (semuaData.length === 0) {
        container.innerHTML = `
            <div class="col-12">
                <div class="alert alert-info">
                    Klik "Muat Data dari API" untuk menampilkan data.
                </div>
            </div>
        `;
        return;
    }

    renderData(semuaData);
});


// =======================
// RENDER DATA
// =======================
function renderData(data) {
    data.forEach(function (karyawan) {
        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';

        col.innerHTML = `
            <div class="card shadow-sm">
                <div class="card-body">
```

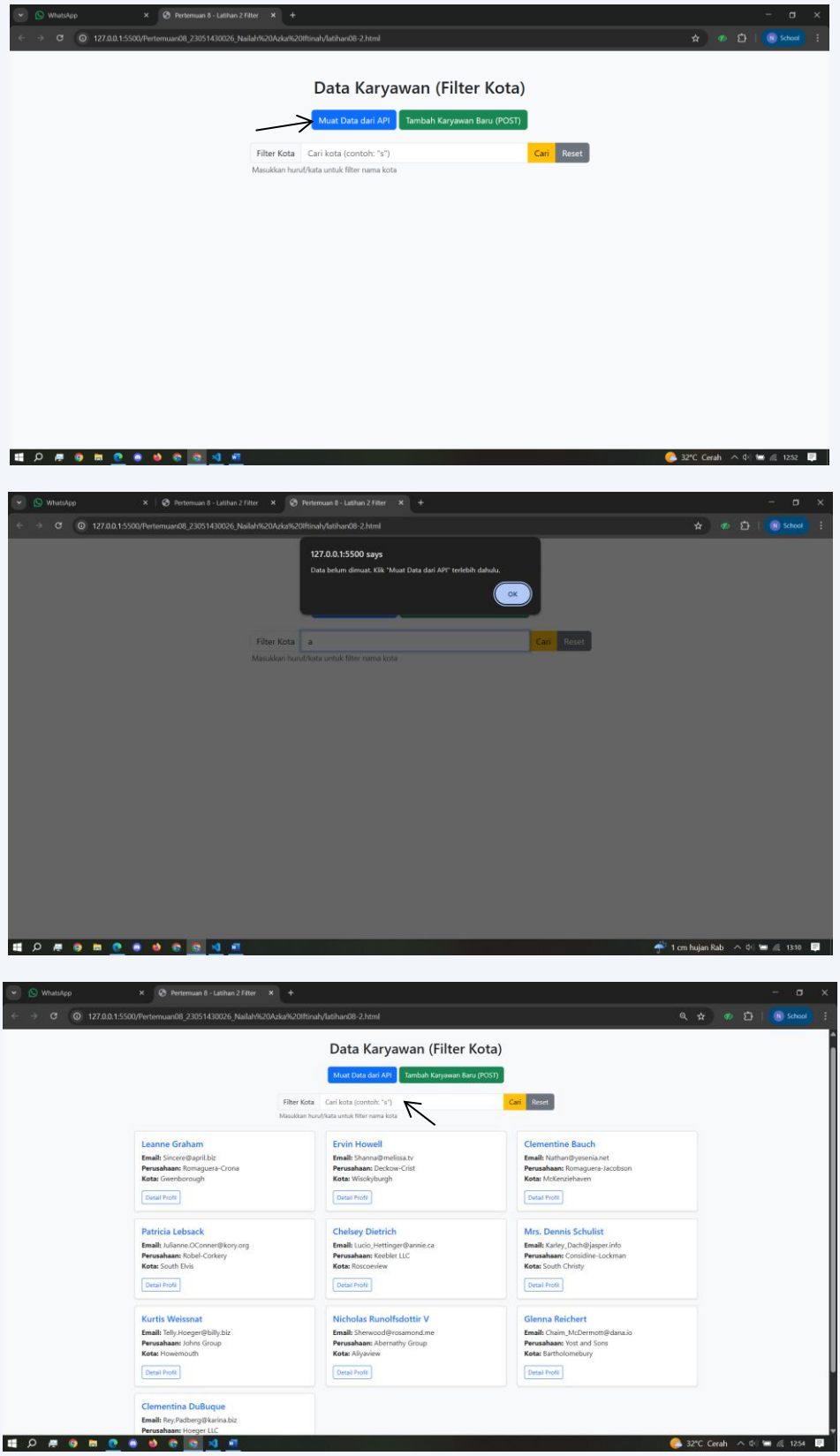
```
                    <h5 class="card-title text-primary">${karyawan.name}</h5>
                    <p class="card-text">
                        <strong>Email:</strong> ${karyawan.email}<br>
                        <strong>Perusahaan:</strong>
${karyawan.company.name}<br>
                        <strong>Kota:</strong> ${karyawan.address.city}
                    </p>
                    <button class="btn btn-outline-primary btn-sm">
                        Detail Profil
                    </button>
                </div>
            </div>
        `;

        container.appendChild(col);
    });
}
```

2. JAWABAN KODE HTML

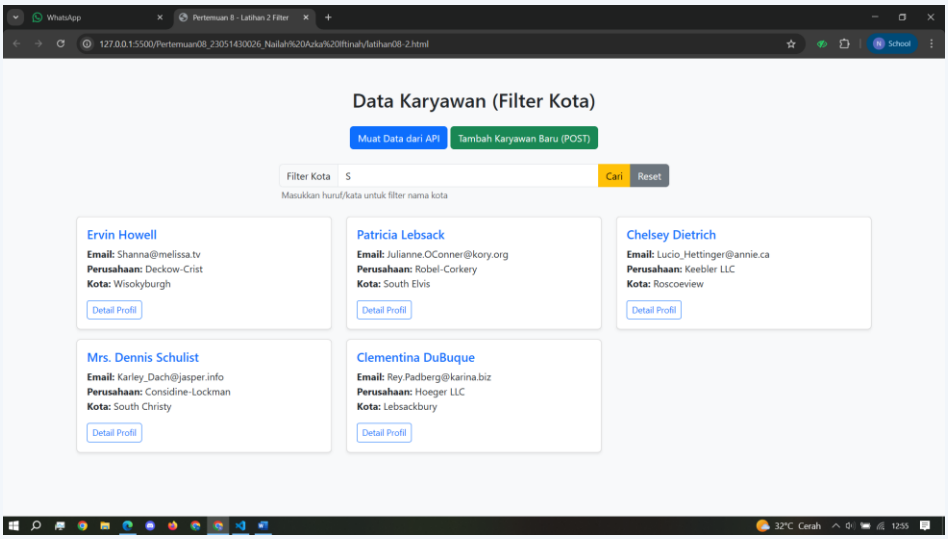
```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Pertemuan 8 - Latihan 2 Filter</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="bg-light">

    <div class="container my-5">
        <h2 class="text-center mb-4">Data Karyawan (Filter Kota)</h2>

        <div class="text-center mb-4">
            <button id="btnLoad" class="btn btn-primary">
                Muat Data dari API
            </button>
            <button id="btnAdd" class="btn btn-success">
                Tambah Karyawan Baru (POST)
            </button>
        </div>

        <div class="row justify-content-center mb-4">
            <div class="col-md-6">
```

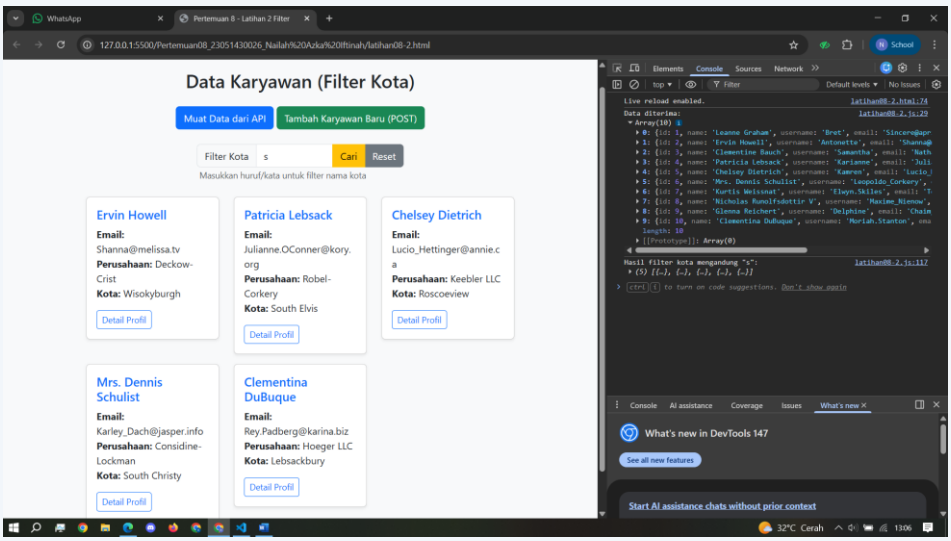
```
                <div class="input-group">
                    <span class="input-group-text">Filter Kota</span>
                    <input type="text" id="inputFilter" class="form-control"
placeholder='Cari kota (contoh: "s")'>
                    <button id="btnFilter" class="btn btn-
warning">Cari</button>
                    <button id="btnReset" class="btn btn-
secondary">Reset</button>
                </div>
                <small class="text-muted ms-1">Masukkan huruf/kata untuk
filter nama kota</small>
            </div>
        </div>

        <div id="loading" class="d-none text-center my-3">
            <div class="spinner-border text-primary" role="status"></div>
            <p>Menghubungi Server...</p>
        </div>

        <div id="containerKaryawan" class="row"></div>
    </div>

    <script src="latihan08-2.js"></script>

</body>
</html>
```

Screenshot Hasil:



Gambar F.2 – Hasil Latihan 2





Gambar F.2 – Hasil Latihan 2

F.4 Tugas Proyek Mini – (Integrasi Data Produksi):

Deskripsi proyek mini:

- Tugas Proyek Mini (Integrasi Data Produksi):
  Anggap API `https://jsonplaceholder.typicode.com/posts` adalah database "Laporan Insiden".
  a. Tampilkan daftar 10 laporan insiden terbaru (Title = Judul Insiden, Body = Deskripsi).
  b. Di setiap card, tambahkan tombol "Tindak Lanjut". Saat diklik, `alert()` menampilkan "Tiket ID sedang diproses oleh Tim Maintenance".

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini ini, pendekatan yang digunakan berfokus pada penerapan konsep Asynchronous JavaScript, Fetch API, JSON, serta integrasi REST API eksternal dalam membangun sistem dashboard laporan insiden produksi. Program dirancang untuk mengambil data dari sumber eksternal yang diasumsikan sebagai database laporan insiden, kemudian menampilkannya ke dalam antarmuka berbasis web secara dinamis.

Tahap awal dilakukan dengan merancang struktur tampilan menggunakan HTML dan Bootstrap. Halaman terdiri dari judul dashboard, identitas pembuat, tombol aksi seperti muat laporan, tambah insiden, dan reset data, serta fitur pencarian dan filter status. Data laporan ditampilkan dalam bentuk card agar mudah dibaca, dan setiap card dilengkapi dengan informasi judul insiden, deskripsi, operator, tanggal, jam, status, serta tombol tindak lanjut. Selain itu, digunakan komponen modal untuk menampilkan detail operator dan informasi tindak lanjut secara lebih rinci. Selanjutnya, logika program diimplementasikan menggunakan JavaScript dengan pendekatan asynchronous. Proses pengambilan data dilakukan menggunakan Fetch API dengan method GET ke endpoint https://jsonplaceholder.typicode.com/posts. Penggunaan async/await memungkinkan program tetap berjalan tanpa menunggu respon server secara blocking, sehingga meningkatkan performa dan pengalaman pengguna. Data yang diterima dari API berbentuk JSON, kemudian diproses menggunakan method .json() untuk dikonversi menjadi object JavaScript.

Data hasil dari API tidak langsung digunakan, melainkan diolah kembali menggunakan method map() untuk menyesuaikan dengan kebutuhan sistem laporan insiden. Setiap data diubah menjadi object baru yang berisi id, judul, deskripsi, informasi operator, status penanganan, tanggal, dan jam. Informasi tambahan seperti operator dan status dibuat secara dinamis menggunakan fungsi terpisah agar struktur data lebih terorganisir. Setelah data siap, informasi ditampilkan ke halaman menggunakan manipulasi DOM melalui fungsi render(). Data disusun dalam bentuk card secara dinamis menggunakan innerHTML. Pada bagian operator, nama tidak ditampilkan secara langsung, melainkan menggunakan label seperti "Operator 1", "Operator 2", dan seterusnya. Ketika label tersebut diklik, sistem akan menampilkan detail lengkap operator melalui modal. Sementara itu, tombol "Tindak Lanjut" digunakan untuk menampilkan informasi penanganan insiden.

Untuk meningkatkan interaktivitas, ditambahkan fitur pencarian dan filter status. Proses ini dilakukan menggunakan method filter() pada array data untuk menyaring laporan berdasarkan kata kunci judul dan status yang dipilih. Hasil penyaringan kemudian ditampilkan kembali ke halaman dengan memanggil fungsi render(). Selain menggunakan data dari API, sistem juga menyediakan fitur penambahan laporan baru melalui form input. Data yang dimasukkan oleh pengguna akan dibentuk menjadi object baru dengan id unik menggunakan Date.now(), kemudian disimpan ke dalam LocalStorage menggunakan JSON.stringify(). Dengan demikian, data tetap tersimpan meskipun halaman di-refresh. Saat aplikasi dijalankan kembali, data tambahan tersebut diambil menggunakan JSON.parse() dan digabungkan dengan data dari API.

Untuk menjaga konsistensi data, disediakan fitur reset yang berfungsi menghapus data tambahan dari LocalStorage menggunakan localStorage.removeItem(), sehingga tampilan kembali ke kondisi awal berdasarkan data dari API. Secara keseluruhan, proyek ini telah menerapkan konsep utama pada materi Pertemuan 8, yaitu penggunaan asynchronous

JavaScript dengan async/await, pemanfaatan Fetch API untuk mengambil data dari REST API eksternal, pengolahan data JSON, serta integrasi data ke dalam tampilan web secara dinamis. Sistem juga dilengkapi dengan fitur interaktif tambahan seperti pencarian, filter, dan penyimpanan lokal yang membuat aplikasi lebih mendekati implementasi nyata di lingkungan industri.

Kode Utama:

1. Jawaban Kode JS

```
class SistemLaporanInsiden {
    constructor() {
        this.URL_API = 'https://jsonplaceholder.typicode.com/posts?_limit=10';

        this.dataTambahan = JSON.parse(localStorage.getItem("dataTambahan")) || []
        this.dataAwal = [];
        this.dataSemua = [];

        this.init();
    }

    init() {
        this.btnMuat = document.getElementById("btnMuatLaporan");
        this.btnSimpan = document.getElementById("btnSimpan");
        this.btnReset = document.getElementById("btnReset");
        this.searchInput = document.getElementById("searchInput");
        this.filterStatus = document.getElementById("filterStatus");
        this.loading = document.getElementById("loading");

        this.btnMuat.onclick = () => this.loadData();
        this.btnSimpan.onclick = () => this.tambahData();
        this.btnReset.onclick = () => this.resetData();
        this.searchInput.oninput = () => this.applyFilter();
        this.filterStatus.onchange = () => this.applyFilter();
    }

    async loadData() {
        this.loading.classList.remove('d-none');

        try {
            const res = await fetch(this.URL_API);
            const data = await res.json();

            this.dataAwal = data.map((_, i) => {
                const tgl = new Date(2026, 0, i + 1);
                const formatTanggal = tgl.toISOString().split("T")[0];
```

```
                return {
                    id: i + 1,
                    judul: [
                        "Mesin Conveyor Berhenti Mendadak",
                        "Kebocoran Oli Mesin Produksi",
                        "Sensor Suhu Tidak Stabil",
                        "Gangguan Listrik Panel",
                        "Overheat Mesin"
                    ][i % 5],
                    deskripsi: "Terjadi gangguan pada proses produksi yang memerlu
penanganan segera.",
                    operator: this.generateOperator(i),
                    status: this.generateStatus(i),
                    tanggal: formatTanggal,
                    jam: (8 + i).toString().padStart(2, '0') + ":00"
                };
            });

            this.dataSemua = [...this.dataTambahan, ...this.dataAwal];
            this.render(this.dataSemua);

        } catch (err) {
            alert("Gagal mengambil data dari API");
        }

        this.loading.classList.add('d-none');
    }

    generateOperator(i) {
        const ops = [
            "Andi Saputra","Budi Santoso","Citra Lestari","Dedi Prasetyo",
            "Eka Wulandari","Fajar Nugroho","Gina Maharani","Hendra Wijaya",
            "Intan Permata","Joko Susilo"
        ];

        return {
            nama: ops[i % ops.length],
            email: ops[i % ops.length].toLowerCase().replace(" ", ".") + "@pabrik.
            perusahaan: "PT Industri Maju",
            kota: "Bekasi",
            shift: ["Pagi","Siang","Malam"][i % 3]
        };
    }

    generateStatus(i) {
```

```
        const s = [
            { nama:"Diproses", warna:"warning", ket:"Insiden sedang dalam proses i
oleh tim maintenance." },
            { nama:"Selesai", warna:"success", ket:"Insiden telah berhasil ditanga
sistem kembali normal." },
            { nama:"Tertunda", warna:"secondary", ket:"Penanganan ditunda karena m
sparepart atau persetujuan manajemen." }
        ];
        return s[i % 3];
    }

    render(data) {
        const konten = document.getElementById("kontenLaporan");
        konten.innerHTML = "";

        data.forEach(l => {
            konten.innerHTML += `
            <div class="col-md-4 mb-3">
                <div class="card">
                    <div class="card-header bg-danger text-white">
                        Laporan No. ${l.id}
                    </div>

                    <div class="card-body">
                        <h5>${l.judul}</h5>
                        <p>${l.deskripsi}</p>

                        <p class="fw-bold text-primary" onclick="showOp(${l.id})"
style="cursor:pointer">
                            Operator: ${l.operator.nama}
                        </p>

                        <p>Tanggal: ${l.tanggal}</p>
                        <p>Jam: ${l.jam}</p>

                        <span class="badge bg-${l.status.warna}">
                            ${l.status.nama}
                        </span>

                        <button onclick="showTin(${l.id})" class="btn btn-danger w
2">
                            Tindak Lanjut
                        </button>
                    </div>
                </div>
            </div>
```

```
            `;
            window["d"+l.id] = l;
        });
    }

    applyFilter() {
        let hasil = this.dataSemua;
        const key = this.searchInput.value.toLowerCase();
        const status = this.filterStatus.value;

        if (key) hasil = hasil.filter(d => d.judul.toLowerCase().includes(key));
        if (status !== "semua") hasil = hasil.filter(d => d.status.nama === status

        this.render(hasil);
    }

    tambahData() {
        const judul = document.getElementById("inputJudul").value;
        const desk = document.getElementById("inputDesk").value;

        if (!judul || !desk) {
            alert("Judul dan Deskripsi wajib diisi");
            return;
        }

        const nama = document.getElementById("selectNama").value ||
                     document.getElementById("inputNama").value;

        const lastId = this.dataSemua.length > 0
            ? Math.max(...this.dataSemua.map(d => d.id))
            : 0;

        const newId = lastId + 1;

        const newData = {
            id: newId,
            judul: judul,
            deskripsi: desk,
            operator: {
                nama: nama,
                email: document.getElementById("inputEmail").value,
                perusahaan: document.getElementById("inputPerusahaan").value,
                kota: document.getElementById("inputKota").value,
                shift: document.getElementById("inputShift").value
            },
            status: this.generateStatus(0),
```

```
            tanggal: document.getElementById("inputTanggal").value,
            jam: document.getElementById("inputJam").value
        };

        this.dataTambahan.unshift(newData);
        localStorage.setItem("dataTambahan", JSON.stringify(this.dataTambahan));

        this.dataSemua.unshift(newData);
        this.render(this.dataSemua);

        this.resetForm();

        const modal = bootstrap.Modal.getInstance(document.getElementById('modalTa
        modal.hide();
    }

    resetForm() {
        document.getElementById("inputJudul").value = "";
        document.getElementById("inputDesk").value = "";
        document.getElementById("selectNama").value = "";
        document.getElementById("inputNama").value = "";
        document.getElementById("inputEmail").value = "";
        document.getElementById("inputPerusahaan").value = "";
        document.getElementById("inputKota").value = "";
        document.getElementById("inputShift").value = "Pagi";
        document.getElementById("inputTanggal").value = "";
        document.getElementById("inputJam").value = "";
    }

    resetData() {
        const yakin = confirm("Apakah Anda yakin ingin mereset semua data?");
        if (!yakin) return;

        localStorage.removeItem("dataTambahan");
        this.dataTambahan = [];
        this.dataSemua = this.dataAwal;
        this.render(this.dataSemua);
    }
}

function showOp(id){
    const d = window["d"+id];
    document.getElementById("isiOperator").innerHTML = `
        <p>Nama: ${d.operator.nama}</p>
        <p>Email: ${d.operator.email}</p>
        <p>Perusahaan: ${d.operator.perusahaan}</p>
```

```
        <p>Kota: ${d.operator.kota}</p>
        <p>Shift: ${d.operator.shift}</p>
    `;
    new bootstrap.Modal(document.getElementById("modalOperator")).show();
}

function showTin(id){
    const d = window["d"+id];
    document.getElementById("isiTindak").innerHTML = `
        <p><strong>Status:</strong> ${d.status.nama}</p>
        <p>${d.status.ket}</p>
        <p>Tiket ID ${id} sedang diproses oleh Tim Maintenance.</p>
    `;
    new bootstrap.Modal(document.getElementById("modalTindak")).show();
}

new SistemLaporanInsiden();
```

2. Jawaban Kode HTML

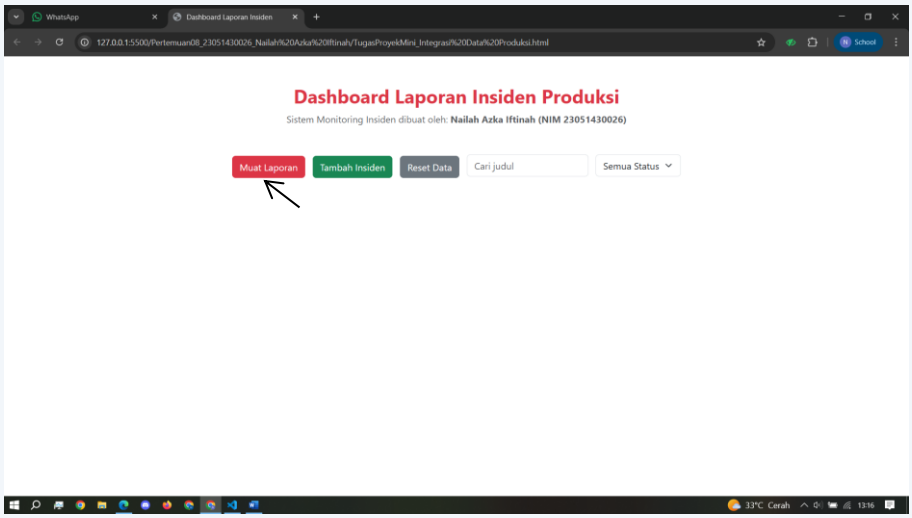
```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Dashboard Laporan Insiden</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.mi
rel="stylesheet">
</head>

<body class="container mt-5">

<div class="text-center mb-5">
    <h2 class="fw-bold text-danger">
        Dashboard Laporan Insiden Produksi
    </h2>
    <p class="text-muted">
        Sistem Monitoring Insiden dibuat oleh:
        <strong>Nailah Azka Iftinah (NIM 23051430026)</strong>
    </p>
</div>

<div class="mb-4 text-center">
    <button id="btnMuatLaporan" class="btn btn-danger">Muat Laporan</button>

    <button class="btn btn-success ms-2" data-bs-toggle="modal" data-bs-
target="#modalTambah">
        Tambah Insiden
```

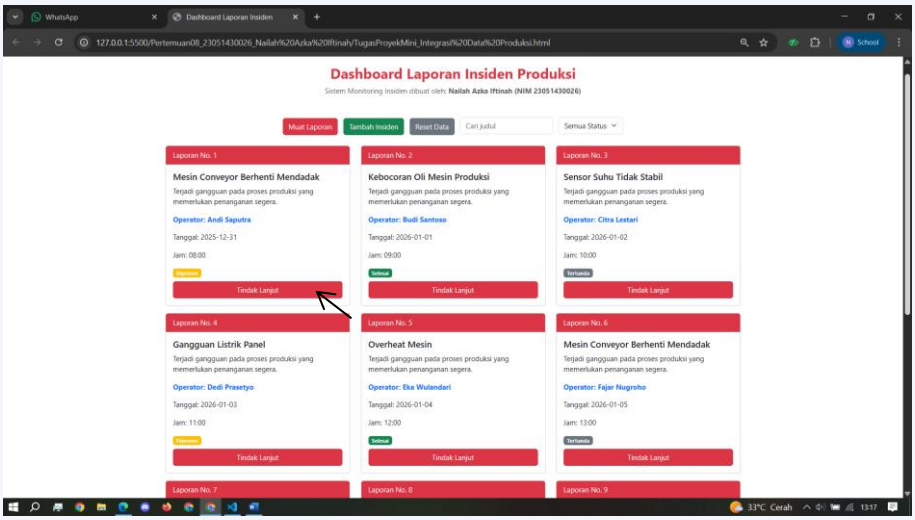
```
    </button>

    <button id="btnReset" class="btn btn-secondary ms-2">
        Reset Data
    </button>

    <input id="searchInput" class="form-control d-inline w-auto ms-2" placeholder=
judul">

    <select id="filterStatus" class="form-select d-inline w-auto ms-2">
        <option value="semua">Semua Status</option>
        <option value="Diproses">Diproses</option>
        <option value="Selesai">Selesai</option>
        <option value="Tertunda">Tertunda</option>
    </select>

    <div id="loading" class="spinner-border text-danger d-none ms-2"></div>
</div>

<div class="row" id="kontenLaporan"></div>

<!-- MODAL OPERATOR -->
<div class="modal fade" id="modalOperator">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header bg-primary text-white">
                <h5>Detail Operator</h5>
                <button class="btn-close" data-bs-dismiss="modal"></button>
            </div>
            <div class="modal-body" id="isiOperator"></div>
        </div>
    </div>
</div>

<!-- MODAL TINDAK LANJUT -->
<div class="modal fade" id="modalTindak">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header bg-danger text-white">
                <h5>Tindak Lanjut Insiden</h5>
                <button class="btn-close" data-bs-dismiss="modal"></button>
            </div>
            <div class="modal-body" id="isiTindak"></div>
        </div>
    </div>
</div>
```

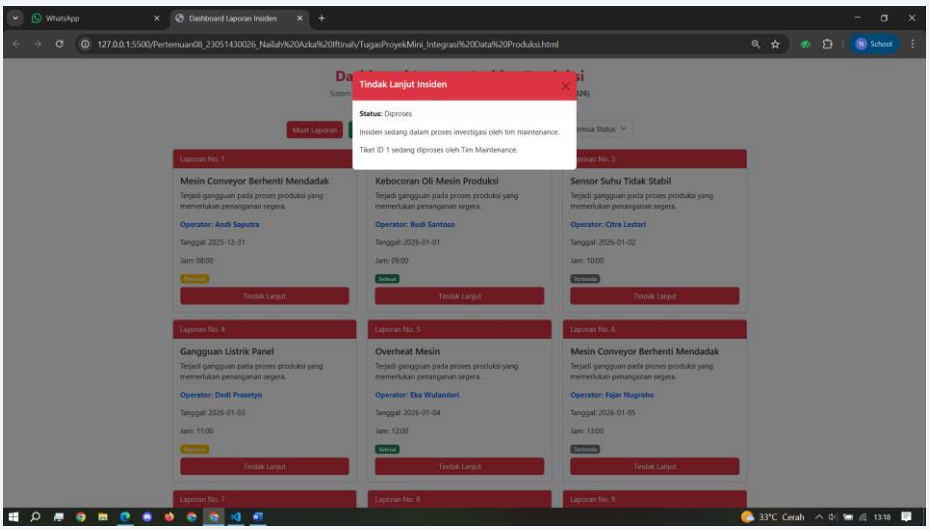
```
<!-- MODAL TAMBAH -->
<div class="modal fade" id="modalTambah">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header bg-success text-white">
                <h5>Tambah Laporan Insiden</h5>
                <button class="btn-close" data-bs-dismiss="modal"></button>
            </div>

            <div class="modal-body">
                <input id="inputJudul" class="form-control mb-2" placeholder="Judu
                <textarea id="inputDesk" class="form-control mb-2"
placeholder="Deskripsi"></textarea>

                <select id="selectNama" class="form-control mb-2">
                    <option value="">Pilih Operator</option>
                    <option>Andi Saputra</option>
                    <option>Budi Santoso</option>
                    <option>Citra Lestari</option>
                    <option>Dedi Prasetyo</option>
                    <option>Eka Wulandari</option>
                    <option>Fajar Nugroho</option>
                    <option>Gina Maharani</option>
                    <option>Hendra Wijaya</option>
                    <option>Intan Permata</option>
                    <option>Joko Susilo</option>
                </select>

                <input id="inputNama" class="form-control mb-2" placeholder="Atau
operator baru">
                <input id="inputEmail" class="form-control mb-2" placeholder="Emai
                <input id="inputPerusahaan" class="form-control mb-2"
placeholder="Perusahaan">
                <input id="inputKota" class="form-control mb-2" placeholder="Kota"

                <select id="inputShift" class="form-control mb-2">
                    <option>Pagi</option>
                    <option>Siang</option>
                    <option>Malam</option>
                </select>

                <input type="date" id="inputTanggal" class="form-control mb-2">
                <input type="time" id="inputJam" class="form-control mb-2">

                <button id="btnSimpan" class="btn btn-success w-100">
```

```
                    Simpan
                </button>
            </div>
        </div>
    </div>
</div>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
<script src="TugasProyekMini_Integrasi Data Produksi.html.js"></script>

</body>
</html>
```

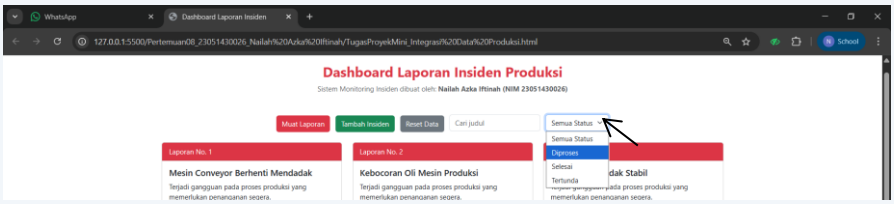
Screenshot Hasil:

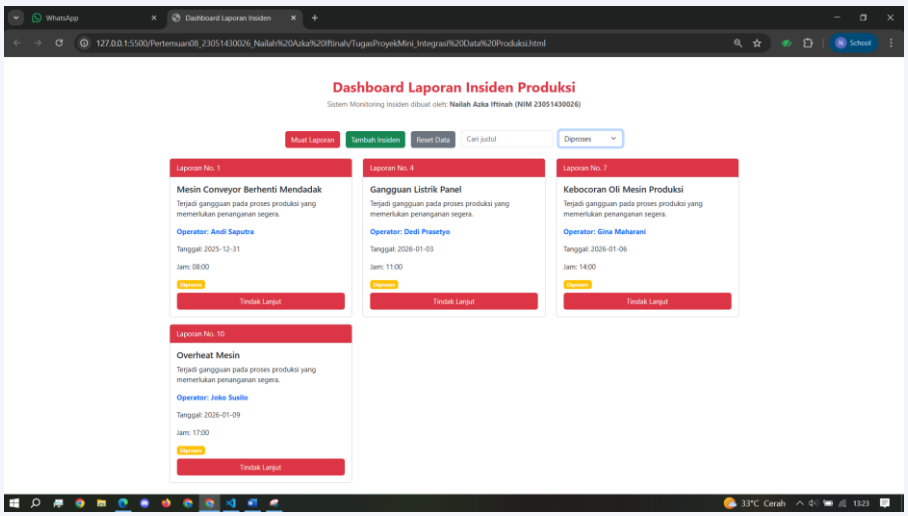


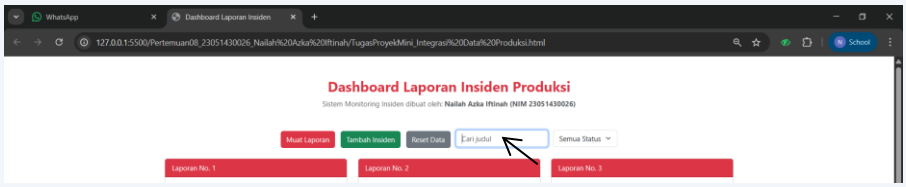


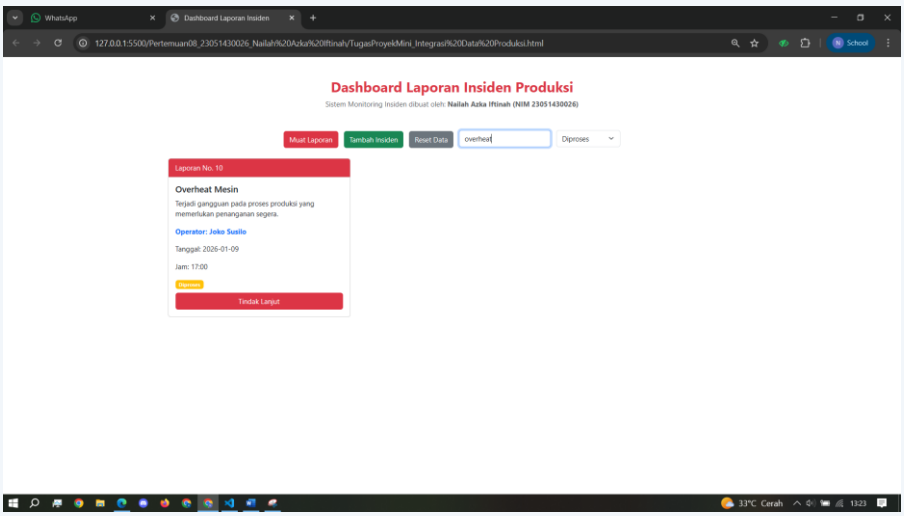


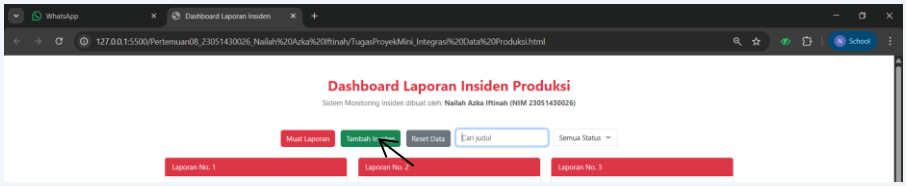
Gambar F.4 – Hasil Proyek Mini



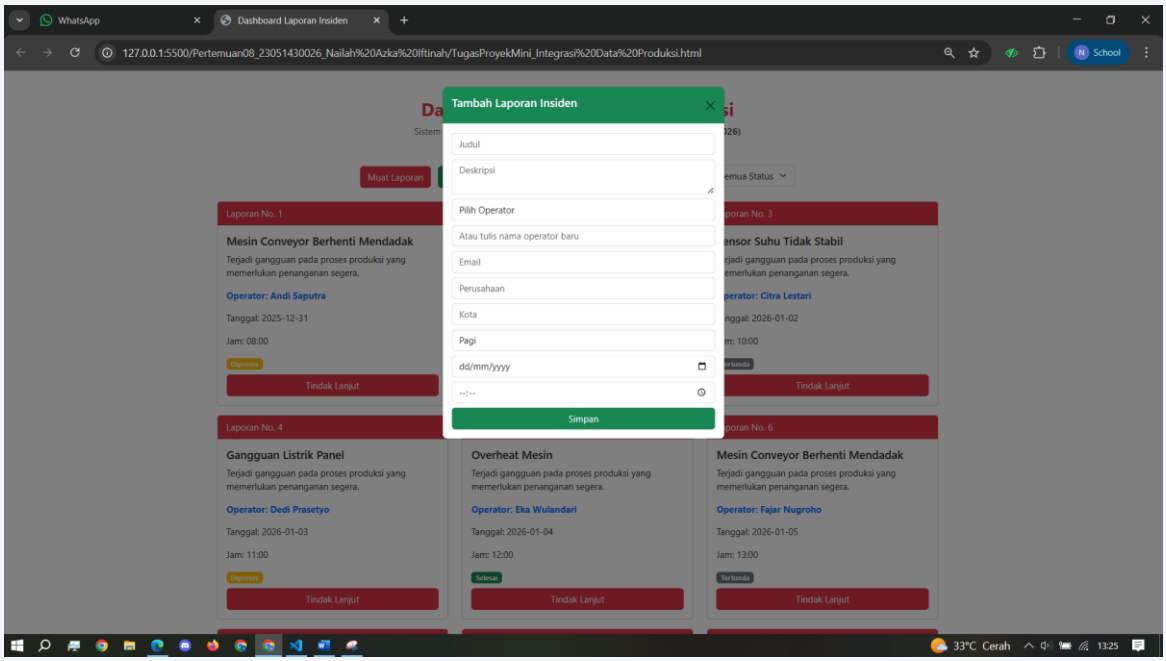


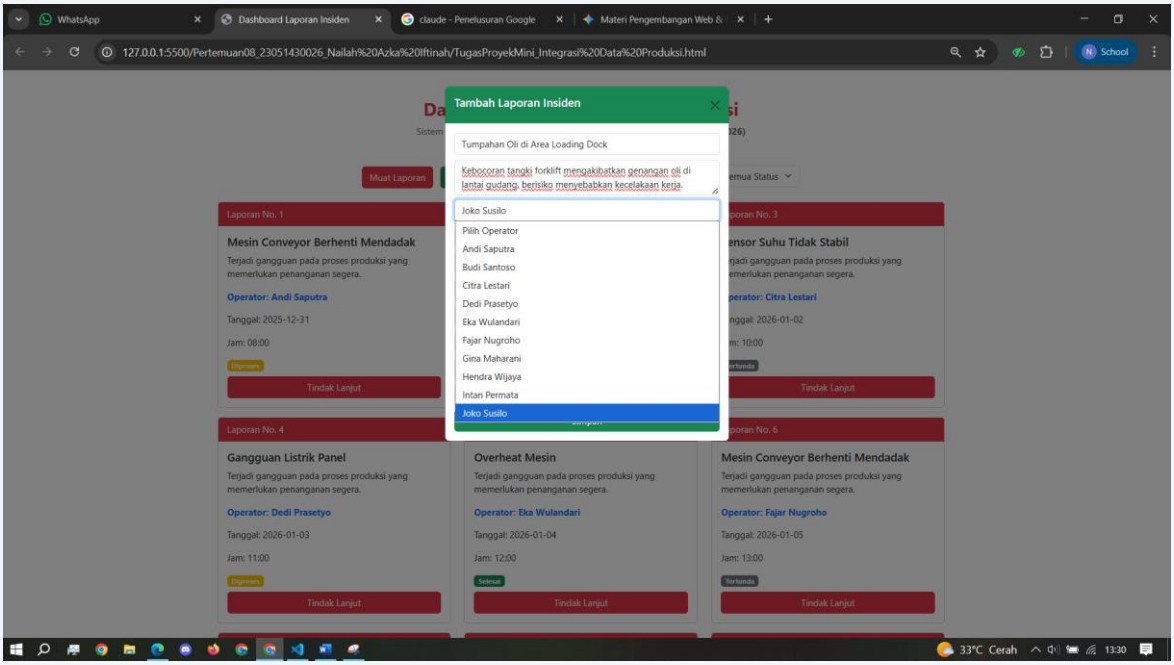




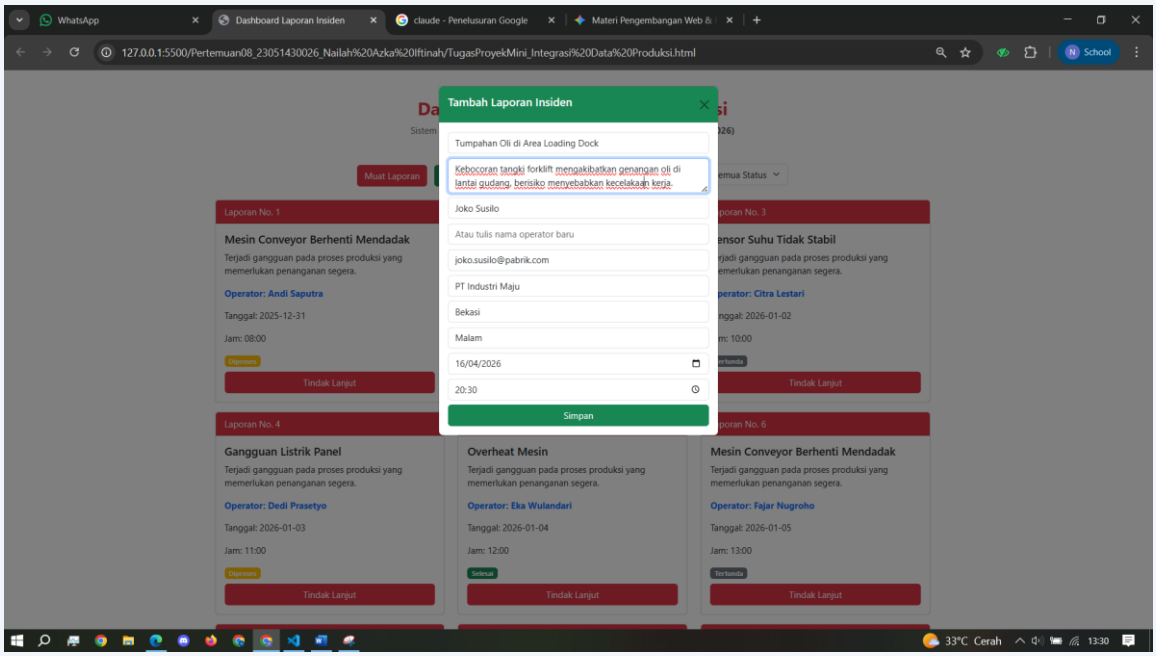


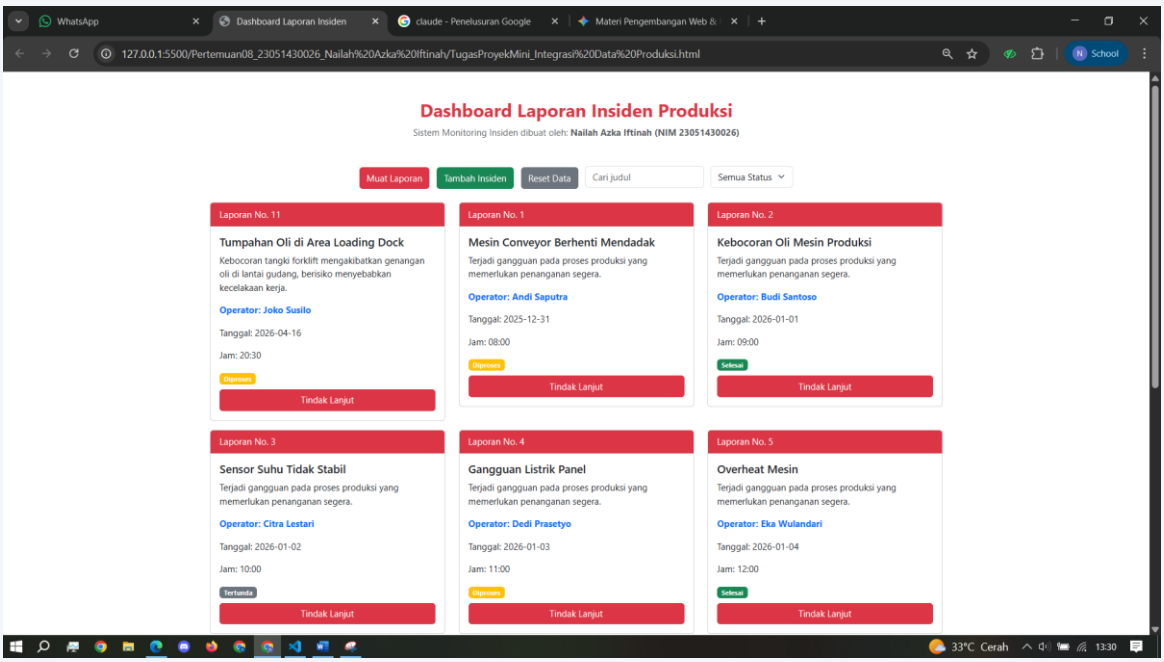
Gambar F.4 – Hasil Proyek Mini



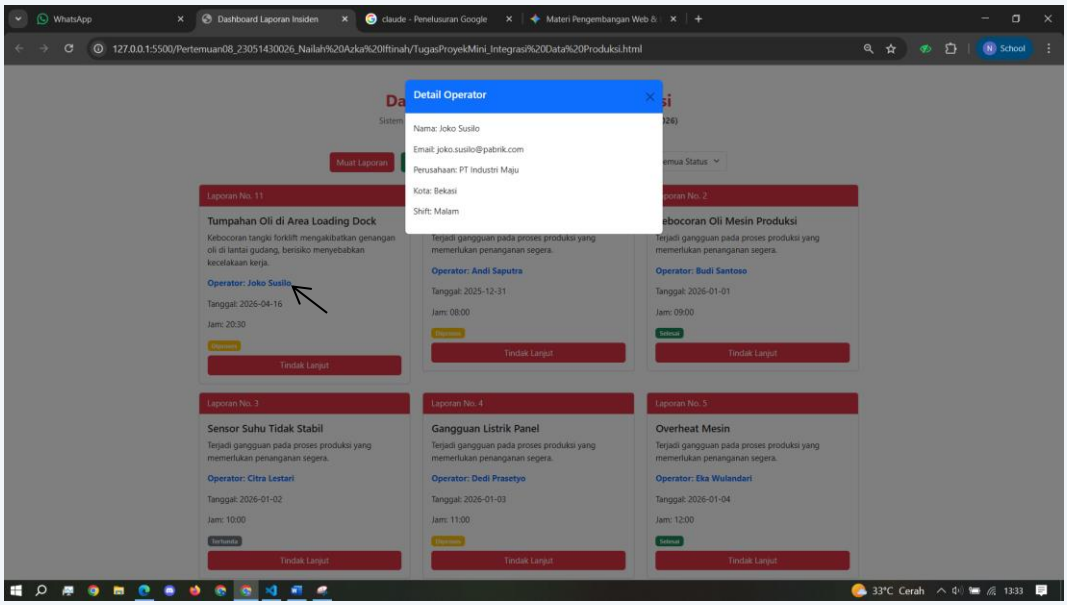


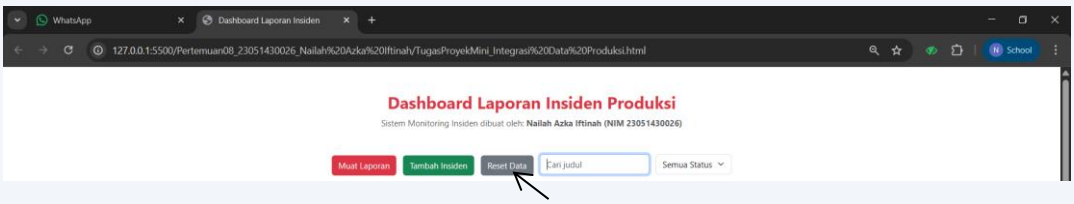
Gambar F.4 – Hasil Proyek Mini

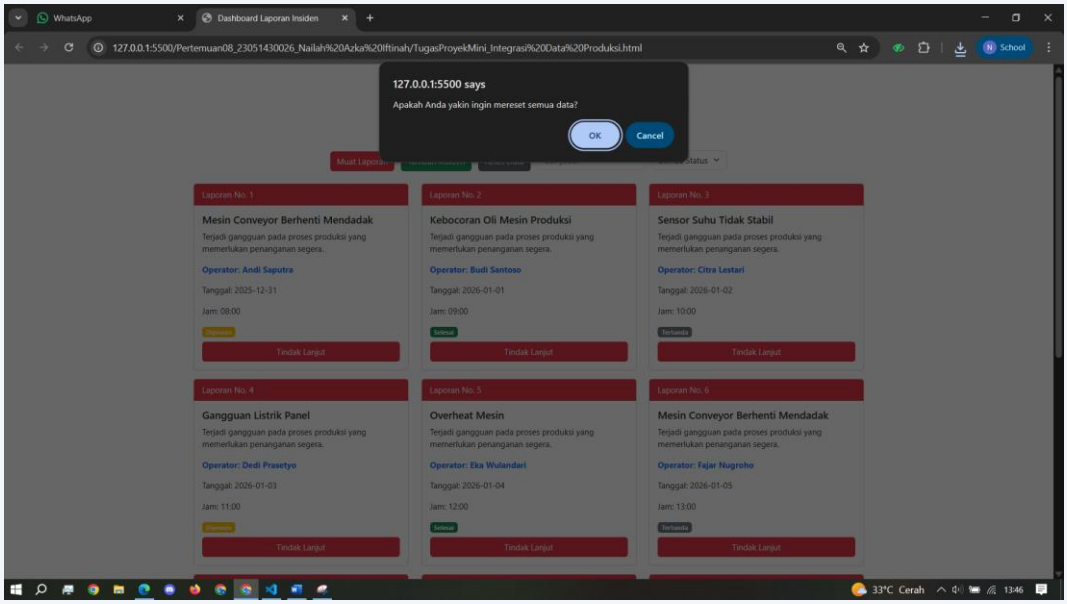




Gambar F.4 – Hasil Proyek Mini







Gambar F.4 – Hasil Proyek Mini

G. PEMBAHASAN

Analisis hasil praktikum secara kritis. Jawab pertanyaan berikut dengan lengkap dan mendalam (bukan hanya "ya/tidak").

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum pertemuan 8 ini secara keseluruhan sudah sesuai dengan teori yang dipelajari tentang konsep asynchronous JavaScript, penggunaan Fetch API, pengolahan data JSON, serta integrasi dengan sumber data eksternal. Dari proses memuat data hingga menampilkan ke dalam dashboard, semua membuktikan bahwa konsep pada modul dapat langsung diterapkan dalam aplikasi nyata berbasis web.

Pada bagian awal praktikum, aplikasi Dashboard Laporan Insiden Produksi dikembangkan dengan memanfaatkan Fetch API untuk mengambil data dari server eksternal. Proses ini dilakukan menggunakan fungsi fetch() yang bersifat asynchronous, sehingga tidak menghambat jalannya aplikasi saat menunggu respon dari server. Hal ini sesuai dengan teori bahwa asynchronous memungkinkan aplikasi tetap responsif meskipun sedang melakukan request data ke server. Penggunaan async/await pada fungsi loadData() juga menunjukkan penerapan konsep modern dalam asynchronous JavaScript. Dengan await, proses pengambilan data dari API dapat ditulis dengan gaya yang lebih mudah dibaca dibandingkan menggunakan callback atau chaining .then(). Selain itu, ditambahkan blok try...catch untuk menangani error jika terjadi kegagalan saat mengambil data dari server, yang sesuai dengan teori bahwa setiap request API harus memiliki mekanisme error handling.

Data yang diperoleh dari API dalam format JSON kemudian diolah menggunakan method .json() untuk dikonversi menjadi object JavaScript. Hal ini sesuai dengan teori JSON (JavaScript Object Notation) yang menyatakan bahwa data dari server umumnya dikirim dalam bentuk JSON dan harus diparsing terlebih dahulu sebelum digunakan dalam aplikasi. Selanjutnya, data hasil fetch diproses menggunakan method .map() untuk disesuaikan dengan kebutuhan sistem, seperti penambahan atribut tanggal, jam, status, dan operator. Ini menunjukkan bahwa data dari API dapat dimodifikasi sebelum ditampilkan, sesuai dengan konsep manipulasi data dalam JavaScript.

Selain menggunakan data dari API, aplikasi juga tetap memanfaatkan LocalStorage untuk menyimpan data tambahan yang diinput oleh user. Data disimpan menggunakan localStorage.setItem() dengan bantuan JSON.stringify() dan dibaca kembali menggunakan JSON.parse(). Hal ini menunjukkan integrasi antara data eksternal (API) dan data lokal (LocalStorage), yang sesuai dengan praktik nyata dalam pengembangan sistem informasi.

Fitur filter pencarian dan filter status juga tetap digunakan dengan memanfaatkan method .filter() pada array. Ini membuktikan bahwa manipulasi data seperti pencarian dan

penyaringan tetap relevan meskipun sumber data berasal dari API. Data yang sudah difilter kemudian ditampilkan kembali menggunakan proses rendering ke dalam DOM.

Pada bagian reset data, ditambahkan konfirmasi menggunakan confirm() sebelum data dihapus dari LocalStorage. Hal ini menunjukkan penerapan aspek user experience (UX) dan validasi tindakan user, sesuai dengan praktik pengembangan aplikasi yang aman dan user-friendly. Secara keseluruhan, praktikum ini menunjukkan bahwa konsep asynchronous JavaScript, Fetch API, JSON, error handling, serta integrasi data eksternal dan lokal dapat diterapkan dengan baik dalam satu aplikasi dashboard. Hasil yang ditampilkan juga membuktikan bahwa aplikasi mampu mengambil, mengolah, menyimpan, dan menampilkan data secara dinamis tanpa mengalami freeze, sehingga sesuai dengan tujuan pembelajaran pada pertemuan 8.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Program tidak berjalan karena beberapa elemen HTML tidak terbaca oleh JavaScript.	Memeriksa kembali kesesuaian antara id pada HTML dengan yang dipanggil di JavaScript agar elemen dapat dikenali dengan benar.

G.3 Kaitan dengan Penerapan di Industri

Materi praktikum pertemuan 8 ini memiliki keterkaitan yang sangat kuat dengan kebutuhan di bidang Teknik Industri, khususnya dalam hal integrasi data, sistem monitoring real-time, serta komunikasi antar sistem berbasis digital. Jika pada pertemuan sebelumnya sistem hanya berfokus pada penyimpanan data secara lokal menggunakan LocalStorage, pada pertemuan ini sistem dikembangkan lebih lanjut dengan kemampuan mengambil data dari sumber eksternal melalui API. Hal ini mencerminkan kondisi nyata di industri, di mana data tidak hanya berasal dari satu sistem, tetapi terdistribusi di berbagai server atau perangkat.

Penggunaan konsep asynchronous dalam praktikum ini sangat relevan dengan sistem industri modern yang membutuhkan kecepatan dan efisiensi. Dalam dunia industri, dashboard produksi sering kali mengambil data dari berbagai sumber seperti sensor mesin, database pusat, atau sistem ERP. Jika proses pengambilan data dilakukan secara synchronous, maka sistem akan mengalami delay atau bahkan freeze saat menunggu respon dari server. Dengan asynchronous JavaScript, sistem tetap dapat berjalan normal tanpa

harus menunggu data selesai dimuat, sehingga user experience tetap terjaga. Hal ini sangat penting dalam monitoring produksi yang membutuhkan respon cepat dan tampilan data yang selalu update.

Penerapan Fetch API dalam praktikum ini juga mencerminkan bagaimana sistem industri melakukan komunikasi data melalui jaringan. Fetch API digunakan untuk melakukan HTTP request ke server eksternal, yang dalam praktik nyata dapat berupa server pusat perusahaan, sistem cloud, atau database produksi. Data yang diambil kemudian ditampilkan dalam bentuk dashboard, serupa dengan sistem monitoring yang digunakan oleh supervisor atau manajer produksi untuk melihat kondisi lapangan secara real-time. Selain itu, penggunaan format JSON dalam pertukaran data merupakan standar yang banyak digunakan dalam industri. Data yang dikirim dari server biasanya berbentuk JSON karena ringan dan mudah diproses oleh berbagai sistem. Dalam praktikum ini, data JSON diubah menjadi object JavaScript untuk kemudian diolah dan ditampilkan. Konsep ini sama dengan proses integrasi data antar sistem di industri, seperti komunikasi antara sistem produksi dengan sistem inventory atau sistem maintenance.

Penanganan error menggunakan try...catch juga memiliki peran penting dalam sistem industri. Dalam kondisi nyata, koneksi ke server bisa saja gagal karena gangguan jaringan atau server down. Oleh karena itu, sistem harus mampu menangani error dengan baik agar tidak menyebabkan crash dan tetap memberikan informasi kepada user. Hal ini mencerminkan pentingnya reliability dalam sistem informasi industri.

Integrasi antara data dari API dan data yang disimpan di LocalStorage juga menggambarkan bagaimana sistem industri menggabungkan data pusat dengan data lokal. Data dari API dapat dianalogikan sebagai data utama dari server pusat, sedangkan LocalStorage sebagai penyimpanan sementara di sisi client. Kombinasi ini memungkinkan sistem tetap fleksibel, misalnya tetap bisa menampilkan data tambahan dari user meskipun data utama berasal dari server. Jika dianalogikan secara sederhana, Fetch API berperan sebagai jalur komunikasi data antar sistem, asynchronous JavaScript sebagai mekanisme agar sistem tetap responsif, JSON sebagai format standar pertukaran data, dan LocalStorage sebagai penyimpanan lokal sementara. Kombinasi dari konsep-konsep ini menghasilkan sistem dashboard yang tidak hanya interaktif, tetapi juga mampu terhubung dengan sumber data eksternal seperti yang digunakan dalam industri modern.

Walaupun implementasi pada praktikum ini masih menggunakan API publik sederhana dan belum terhubung dengan sistem industri sebenarnya, konsep yang dipelajari sudah memberikan gambaran yang sangat jelas tentang bagaimana sistem monitoring produksi,

dashboard operasional, dan integrasi data bekerja di dunia nyata. Bagi mahasiswa Teknik Industri, pemahaman ini sangat penting karena akan sering digunakan dalam sistem seperti ERP, Manufacturing Execution System (MES), maupun dashboard monitoring berbasis web yang digunakan dalam pengambilan keputusan di industri.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Mahasiswa mampu memahami konsep asynchronous JavaScript sehingga program tetap berjalan tanpa menunggu proses lain selesai.
2	Mahasiswa mampu menggunakan Fetch API untuk mengambil data dari sumber eksternal (API).
3	Mahasiswa memahami format JSON serta mampu mengolah data menjadi object JavaScript.
4	Mahasiswa mampu menerapkan async/await dan menangani error menggunakan try-catch agar sistem tetap stabil secara cepat dan efisien, sesuai dengan kebutuhan pengambilan keputusan di industri.
5	Mahasiswa mampu mengintegrasikan data API dengan data lokal serta mengembangkan kemampuan problem solving dan kreativitas dalam pembuatan dashboard, serta meningkatkan kedisiplinan dalam menyelesaikan praktikum dan laporan tepat waktu.

I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Shofiana, D. A., Mardiansyah, S. S., Parabi, M. I., & Syarif, A. (2024). Implementasi REST API menggunakan JSON Web Token (JWT) pada sistem monitoring KPI berbasis mobile. Jurnal Komputasi.
[3]	Pattinama, Y. L., Ferdiansyah, F., Susanti, I., & Painem. (2023). Implementasi REST API web service dengan otentifikasi JSON Web Token untuk aplikasi properti. Jurnal Informatik.
[4]	Mozilla Developer Network (MDN). Fetch API. https://developer.mozilla.org/ dan JSON.org. Introducing JSON. https://www.json.org/
[5]	Putra, F. P. E., Efendi, R. W., Tamam, A. B., & Pramadi, W. A. (2025). Tren dan praktik terbaik dalam pengembangan web berbasis API

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	**Aspek Penilaian**	**Bobot (%)**	**Nilai (0-100)**	**Skor Akhir**	**Catatan Penilai**
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
	TOTAL NILAI AKHIR	**100%**			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 _____ Nailah Azka Iftinah NIM:23051430026	_____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____